PhyAuth-RF: Physical-Layer Device Authentication of 433 MHz Transmitters via RF Timing Fingerprints, Edge TinyML and a Two-Factor Smart Lock

PROJECT REPORT

Bachelor of Technology

School of Electronics Engineering (SENSE)

BECE306P – Digital Communication Systems

by

Pratik Kumar (24BML0027)

Nikunj Goydani (24BEC0338)

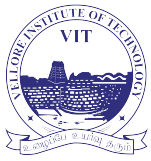
Shruti (24BML0033)

Uday Krishan Khandelwal (24BML0017)

Under the guidance of

Dr. NandaKumar

School of Electronics Engineering (SENSE)



VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

October 2026

Abstract

Low-cost 433 MHz remote controls (garage doors, gates, alarm panels, smart plugs) transmit a fixed 24-bit code with on–off keying. Anyone who records the code can replay it, because the receiver authenticates the *message*, never the *transmitter*. This project, **PhyAuth-RF**, authenticates the physical transmitter instead, using only the hardware a student can buy for about Rs. 2,000: two ESP32 boards, an SRX882 superheterodyne receiver, an STX882 transmitter and a QC5681 two-button EV1527 remote.

The receiver ESP32 timestamps every edge of the demodulated signal with its 240 MHz cycle counter (4.17 ns resolution). A distortion-invariant clock-recovery method (pair period and 31T sync gap) decodes the EV1527 frame even when the SRX882 slicer shortens every HIGH by up to a third of a symbol, and eleven code-independent timing features are extracted per frame. A 963-parameter EdgeMLP maps the features to a 16-dimensional embedding, and an open-set Mahalanobis authenticator with Ledoit–Wolf covariance accepts or rejects each press on-chip, fusing four frames. A larger 21 k-parameter host model (1D ResNet + MLP) runs on the laptop in parallel, and a FastAPI + Next.js dashboard records data, trains, exports the model as a 7 KB C header and visualises every decision.

On held-out real captures the deployed on-chip model reached 100 % closed-set accuracy, a frame-level EER of 0.49 % (AUC 0.99996) and a burst-level EER of 0 %, with **all 30 replay bursts of a code-cloning transmitter and all 30 bursts of an unknown transmitter rejected**, and no genuine burst rejected. Fusing four frames cut the genuine rejection rate from 45.6 % (single frame) to 1.1 % at a decision latency of about 172 ms. On top of the fingerprint, a two-factor **smart lock** runs on the ESP32: the door opens only when the enrolled remote (something you *have*, verified by its RF fingerprint) enters a secret ON/OFF press pattern (something you *know*), with lockout after three failures, learn mode, flash persistence, a green status LED and a buzzer. The report documents the design, the measured limits of the hardware (reliable window 0.75–1.25 m), the engineering problems solved, and the alignment of the work with the UN Sustainable Development Goals.

Keywords: RF fingerprinting, physical-layer authentication, OOK/ASK, EV1527, ESP32, TinyML, Mahalanobis distance, open-set recognition, replay attack, two-factor smart lock.

Contents

Abstract **2**
List of Figures **4**
List of Tables **4**
List of Abbreviations **5**
Chapter 1: Introduction & Literature Survey **6**
1.1 Background 6
1.2 Literature Survey 6
1.3 Research Gap 7
1.4 Project Objectives 7
1.5 Scope and Organisation of the Report 8
Chapter 2: Conceptual Framework & Methodology **9**
2.1 Problem Formulation 9
2.2 Signal Model: EV1527 On–Off Keying 9
2.3 System Architecture 10
2.4 Hardware Design 11
2.5 Receiver Firmware: Nanosecond Edge Capture 13
2.6 Pulse DSP: Distortion-Invariant Clock Recovery and Decoding 14
2.7 Physical Fingerprint Features 14
2.8 Learning: Embedding Models 15
2.9 Open-Set Authenticator and Burst Fusion 15
2.10 Edge Deployment 16
2.11 Two-Factor Smart Lock 16
2.12 Software Stack and Dashboard 17
2.13 Experimental Methodology 17
2.14 Implementation Plan 18
Chapter 3: Innovation & Feasible Solution **19**
3.1 Novel Aspects of the Solution 19
3.2 Comparison with Conventional Approaches 19
3.3 Technical Feasibility 19
3.4 Economic Feasibility 20
3.5 Safety Considerations 20
3.6 Operational Constraints 20
Chapter 4: Results & Discussion **21**
4.1 Physical Prototype 21
4.2 Dataset 21
4.3 Feature Space 21
4.4 Authentication Performance 23
4.5 Distance Robustness 24
4.6 Burst Fusion and Latency 25
4.7 Cross-Session Check and Liveness 25
4.8 Live System and Dashboard 26
4.9 Smart Lock Demonstration 27
4.10 Comparative Benchmarking 28
4.11 Engineering Problems Solved 29
4.12 Limitations and Threats to Validity 29
Chapter 5: Sustainable Development Goals (SDGs) & Societal Impact **30**
5.1 Alignment with the UN Sustainable Development Goals 30
5.2 Benefits to Society 30
5.3 Environmental and Ethical Considerations 30
Chapter 6: Conclusion & References **31**
6.1 Conclusion 31
6.2 Future Work 31
6.3 References 31
Appendix A: Pin Map, Commands and Software Organisation **33**

List of Figures

Figure 2.1: A real FOB_A frame captured by the receiver at 1 m (top) and the per-pulse timing residual relative to the recovered grid (bottom). The systematic HIGH/LOW bias is receiver distortion; the small pulse-to-pulse variation on top of it carries the fingerprint. 10
Figure 2.2: Distribution of all FOB_A pulse widths in units of the recovered clock T (about 20 000 pulses). HIGH and LOW symbols form separate, shifted clusters around 1T and 3T. 10
Figure 2.3: System architecture of PhyAuth-RF: transmitters, the receiver/verifier node (all decisions on-chip) and the laptop tool-chain (record, train, export, visualise). 11
Figure 2.4: Receiver / verifier node (labelled): ESP32 DevKit, SRX882 receiver with helical antenna, green status LED with 470 Ω resistor, and piezo buzzer. 12
Figure 2.5: Transmitter / emulator node (labelled): ESP32-S3-DevKitC (N16R8) driving an STX882 ASK transmitter with a helical antenna. 12
Figure 2.6: Genuine device FOB_A: QC5681 two-button EV1527 remote (front view). 13
Figure 2.7: Circuit schematic of the two nodes. The SRX882 is powered from 3.3 V so that its DATA output is a safe logic level for the ESP32. 13
Figure 2.8: Flowchart of the processing chain from a radio edge to an authentication decision (identical in the ESP32 C code and in Python). 14
Figure 2.9: State machine of the two-factor pattern lock running on the receiver ESP32. 17
Figure 4.1: Frames that passed the quality gate, by transmitter and distance (FOB_A_OFF includes 131 frames recorded on 21 Oct to strengthen the OFF button). 21
Figure 4.2: Raw physical features of every accepted frame. Left: clock period vs. bit-period jitter (13 of 1 442 frames that locked to 3T lie off-scale). Right: short-HIGH duty error vs. first-edge residual; the clone (red) is clearly apart from the genuine remote (purple). 22
Figure 4.3: Dashboard Model lab: feature importance and PCA of the embedding space (previous model version 20261021-213950). Enrolled devices form tight clusters; the clone (×, not enrolled) lands between FOB_A and the emulators, outside every acceptance region. 23
Figure 4.4: Dashboard Model lab: ROC on a logarithmic false-accept axis and the closed-set confusion matrix with per-distance frame FRR (previous model version, 213950). 24
Figure 4.5: (a) Single-frame genuine rejection by distance for both models. (b) Decoding window of the SRX882 rig observed on the bench. 24
Figure 4.6: Genuine rejection rate and decision latency against the number of frames fused per press (90 genuine presses). 25
Figure 4.7: Live authentication: genuine remote accepted on the ESP32 (score 0.91, 4 frames, T = 334.64 µs), with the last frame's pulse train and timing residuals. 26
Figure 4.8: Live authentication: a borderline EMU_1 press (score 1.08 > 1) flagged as spoof suspected – an example of a false rejection. 26
Figure 4.9: Decision log: verdict, device, code, score, frames and which decider (ESP32 or laptop) made the decision. 27
Figure 4.10: Smart Lock workspace, locked: padlock, progress dots for the 9-press pattern, the two factor cards and the settings stored in the ESP32's flash. 27
Figure 4.11: Smart Lock workspace after the correct pattern on the genuine remote: UNLOCKED, auto-relock countdown, factor 1 'genuine' (score 0.32). 28

List of Tables

Table 1.1: Summary of related work 7
Table 2.1: Hardware used 11
Table 2.2: The eleven code-invariant timing features 15
Table 2.3: Transmitters used in the experiments 17
Table 2.4: Implementation plan 18
Table 3.1: Qualitative comparison 19
Table 3.2: Bill of materials (approximate Indian retail prices, October 2026) 20
Table 4.1: Permutation importance of the features for the edge model (test set) 22
Table 4.2: Results of the final model (version 20261021-224341) on held-out data 23
Table 4.3: Leave-one-distance-out test (presses; development model trained on 17 Oct data) 24
Table 4.4: Presses from a later session (21 Oct) against a model trained on 17 Oct data 25
Table 4.5: Start-up drift per press (liveness profiles) 25
Table 4.6: Unit tests of the lock state machine (all 35 project tests pass) 28
Table 4.7: Problems found during development and their fixes 29
Table 5.1: Mapping to SDGs 30
Table A.1: Pin map 33
Table A.2: Main serial commands of the receiver firmware 33
Table A.3: Repository layout 33

List of Abbreviations

Abbreviations and symbols

Abbreviation	Meaning
ASK / OOK	Amplitude-shift keying / on–off keying
AUC	Area under the ROC curve
CE	Cross-entropy loss
EER	Equal error rate (FAR = FRR)
EV1527	One-time-programmable 20-bit-address remote-control encoder IC
FAR / FRR	False accept rate / false reject rate
FSM	Finite-state machine
ISR	Interrupt service routine
LW	Ledoit–Wolf covariance shrinkage
MAC	Multiply–accumulate operation
MLP	Multi-layer perceptron
NVS	Non-volatile storage (ESP32 flash key–value store)
PLA / RFFI	Physical-layer authentication / RF fingerprint identification
RMT	ESP32 remote-control (pulse) peripheral
ROC	Receiver operating characteristic
SDG	UN Sustainable Development Goal
SDR	Software-defined radio
T	Elementary symbol period of the encoder (about 335 µs for FOB_A)
TinyML	Machine learning on microcontrollers

CHAPTER 1

Introduction & Literature Survey

1.1 Background

Sub-GHz remote controls are among the most widely deployed wireless devices. A typical 433.92 MHz key fob contains a single encoder chip (EV1527, PT2262 or HT12E), a SAW-stabilised oscillator and a coin cell. When a button is pressed, the encoder emits a short burst of identical frames by switching the carrier on and off (OOK). The receiver, often a $1 superheterodyne module, slices the envelope back into a logic waveform, decodes the 24-bit code and drives a relay if the code is known.

Security therefore rests entirely on the secrecy of a 20-bit address that is broadcast in clear every time the button is pressed. Capturing it needs nothing more than a receiver of the same kind; replaying it needs a transmitter module costing less than the remote itself. Even rolling-code systems have been defeated by jam-and-replay attacks [10], [11]. The root cause is architectural: the receiver checks *what* was sent, but not *who* sent it.

Every physical transmitter, however, imprints small analogue imperfections on its signal: the exact frequency of its RC or crystal timebase, its jitter, the rise and fall behaviour of its power amplifier, and the way its oscillator settles after power-up. These imperfections are not part of the code and are hard to copy with a different circuit. Measuring them to identify the device is called **RF fingerprinting** or **physical-layer authentication** [4], [9], [27].

Digital Communication Systems (BECE306P) treats the transmitter as an ideal modulator and the channel as the only source of impairment [21]. This project turns that view around: the *transmitter's* impairments become the signal of interest, and the receiver's own distortion becomes the main obstacle to measuring them. It combines line coding (EV1527 pulse-width coding), synchronisation (symbol-clock recovery), detection theory (one-class hypothesis testing with FAR/FRR trade-offs) and embedded implementation.

1.2 Literature Survey

Table 1.1 summarises the work that most directly shaped this project. Early research showed that devices can be identified remotely from clock skew [1], [28] and from the turn-on transient of the carrier [3], [29]. PARADIS [2] used modulation-domain errors of 802.11 cards measured with a vector signal analyser. Deep-learning approaches since 2018 learn the fingerprint directly from raw I/Q samples [5]–[8]; surveys [4], [9], [27] note that nearly all of this work uses laboratory SDRs with high-rate ADCs.

Table 1.1: Summary of related work

Ref.	Signal / platform	Method	Relevance and limitation for this project
[1] Kohno 2005	TCP timestamps over the Internet	Clock-skew estimation	Timebase error identifies a device; needs long observation
[28] Jana 2010	802.11 beacons	Clock skew of access points	Skew detects rogue APs; needs synchronised timestamps
[3] Danev 2009	IEEE 802.15.4 nodes, high-rate oscilloscope	Turn-on transient features	Very low EER, but GS/s capture equipment
[29] Ureten 2007	Transient of RF bursts	Envelope transient + classifier	Shows transients are identifying; analogue front end needed
[2] Brik 2008 (PARADIS)	802.11b NICs, vector signal analyser	Modulation errors + classifier	High accuracy; lab-grade instrument
[5] Merchant 2018	ZigBee, SDR I/Q	CNN on error signal	Deep learning on I/Q; SDR + GPU
[6], [7] Riyaz, Sankhe	Wi-Fi radios, USRP SDR	CNN (ORACLE)	Robust to channel; costly SDR testbed
[8] Shen 2022	LoRa, SDR	Spectrogram CNN, channel-robust	Addresses channel drift; SDR required
[10] Garcia 2016	Automotive RKE	Cryptanalysis + replay	Motivates hardware-level authentication
[14] Lee 2018	Deep features	Mahalanobis OOD detection	Basis for our open-set (unknown-device) rejection
[17], [18] TinyML	Microcontrollers	Quantised small networks	Feasibility of on-device inference

Three observations follow from the survey. First, timing-domain fingerprints (clock skew, jitter, transient timing) are physically meaningful and survive envelope detection, so they can be measured *without* I/Q sampling. Second, open-set rejection is essential: a closed-set classifier always assigns an unknown or cloned transmitter to one of the enrolled classes; distance-based methods [13], [14], [26] with metric-learning embeddings [15], [16] avoid this. Third, microcontroller-class inference is mature [17], [18], but we did not find RF fingerprinting demonstrated end-to-end on a $5 MCU with a $1 OOK receiver.

1.3 Research Gap

- Existing RFFI systems rely on SDRs or oscilloscopes; the receiver itself usually costs more than 100 times the device it protects.
- Cheap superheterodyne OOK receivers heavily distort pulse widths (we measured up to 0.33 T); published decoders and fingerprinting methods do not account for this.
- Few works close the loop from capture to an *on-device* accept/reject decision that drives an actuator, with reproducible training and deployment.
- Physical fingerprints are usually studied as a stand-alone factor; combining them with a user secret on the same unmodified remote has not been explored for low-cost access control.

1.4 Project Objectives

1. Decode EV1527 frames reliably from a real QC5681 remote through the distortion of an SRX882 receiver, at a usable distance.
2. Extract code-independent timing features that identify the physical transmitter, with nanosecond-resolution timestamps on an ESP32.

3. Train an embedding model and an open-set authenticator that accepts the enrolled remote and rejects (a) a transmitter replaying its exact code and (b) unknown transmitters.
4. Run the authenticator on the ESP32 itself (TinyML, < 8 KB) and keep it bit-exact with the Python implementation.
5. Build a two-factor smart lock on the receiver: RF fingerprint (have) + secret ON/OFF press pattern (know), with lockout and status indication (LED, buzzer).
6. Provide a full-stack tool (recorder, trainer, exporter, live dashboard) and evaluate the system honestly on real hardware, including its limits.

1.5 Scope and Organisation of the Report

The work is limited to the hardware listed in Chapter 2: one genuine remote (two buttons), one receiver node and one ESP32-S3 + STX882 node used to emulate further transmitters and attacks. Chapter 2 develops the framework and methodology, Chapter 3 the innovations and feasibility, Chapter 4 the results, Chapter 5 the SDG alignment and Chapter 6 concludes. Appendices give the pin map, commands and the software layout.

CHAPTER 2

Conceptual Framework & Methodology

2.1 Problem Formulation

A press of a transmitter produces a burst of frames x_1, x_2, …. Each frame carries a code that *claims* an identity c (the enrolled device that owns that code). The receiver must decide between

$$H_0 : \mathbf{x} \sim \text{device } c \qquad \text{versus} \qquad H_1 : \mathbf{x} \sim \text{any other transmitter} \tag{2.1}$$

without ever seeing examples of H_1 during training (an attacker's hardware is unknown in advance). This is a one-class, open-set verification problem. The design goals are a false-accept rate of zero for code clones and unknown devices, a genuine false-reject rate of a few percent per press, a decision within a fraction of a second, and computation that fits a microcontroller. A second problem, the smart lock, is a sequential decision over presses: open only if the pattern is correct *and* every press passed H_0.

2.2 Signal Model: EV1527 On–Off Keying

The EV1527 encodes 20 address bits and 4 key bits (24 bits). Every bit lasts 4T: a '0' is 1T HIGH + 3T LOW and a '1' is 3T HIGH + 1T LOW. Each frame ends with a sync symbol of 1T HIGH followed by 31T LOW, and frames repeat while the button is held [24]. For the QC5681 remote the ON button gives code 0x936768 (key nibble 8) and OFF gives 0x936764 (key nibble 4); the measured symbol period is $T \approx 335$ µs, so

$$T_{frame} = 24 \cdot 4T + 32T = 128\,T \approx 42.9 \text{ ms} \tag{2.2}$$

The SRX882 is a superheterodyne receiver with an envelope detector and a data slicer. Its slicer threshold lags the envelope, so every HIGH is shortened and every LOW lengthened by an amount δ that depends on signal strength. With per-pulse noise ε_i the received widths are

$$w_i^H = u_i T - \delta + \varepsilon_i, \qquad w_i^L = u_i T + \delta + \varepsilon_i, \qquad u_i \in \{1, 3, 31\} \tag{2.3}$$

Figure 2.1 shows a real frame of the genuine remote: every HIGH is 10–17 % short and every LOW 10–17 % long. Close to the antenna δ reaches 0.33T, and then a 3:1 / 1:3 pattern at T fits a 4:2 / 1:5 grid at 2T/3 almost equally well, which made the first, conventional decoder lock onto the wrong clock.

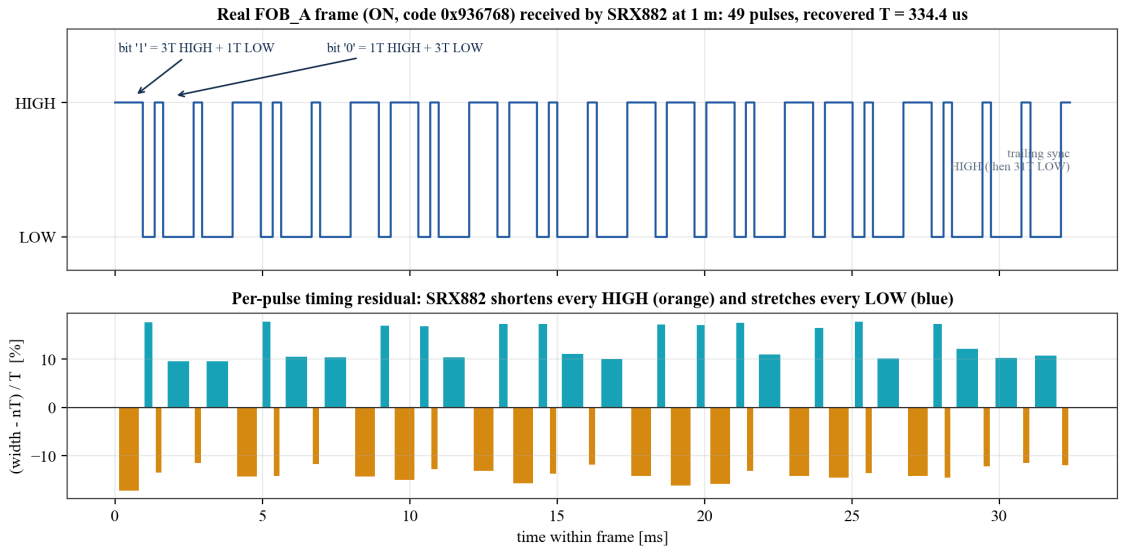


Figure 2.1: A real FOB_A frame captured by the receiver at 1 m (top) and the per-pulse timing residual relative to the recovered grid (bottom). The systematic HIGH/LOW bias is receiver distortion; the small pulse-to-pulse variation on top of it carries the fingerprint.

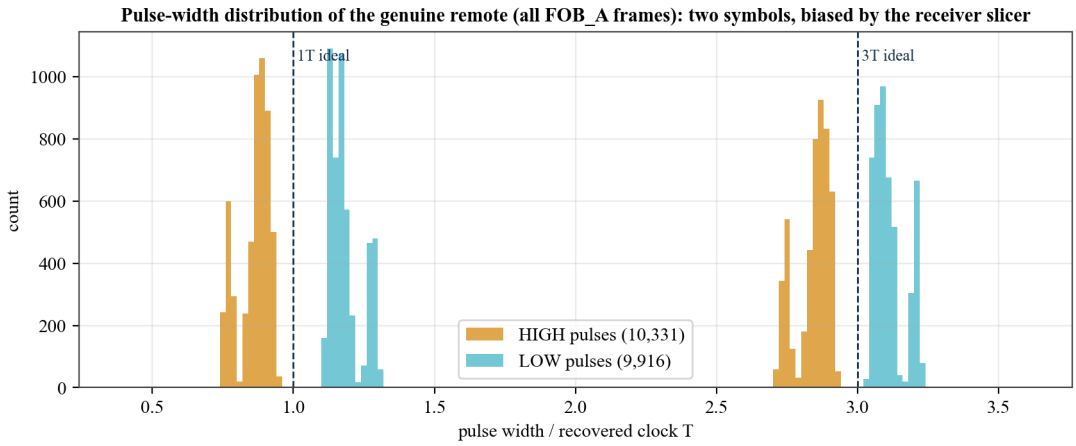


Figure 2.2: Distribution of all FOB_A pulse widths in units of the recovered clock T (about 20 000 pulses). HIGH and LOW symbols form separate, shifted clusters around 1T and 3T.

2.3 System Architecture

Figure 2.3 shows the complete system. Transmitters (the genuine remote and the ESP32-S3 emulator) send OOK at 433.92 MHz. The receiver node captures edges, decodes, extracts features and authenticates on-chip; the same frames are streamed as JSON lines to the laptop, where they are recorded, used for training and decided a second time by the larger host model. The trained edge model is exported as C arrays and flashed back to the ESP32, closing the loop.

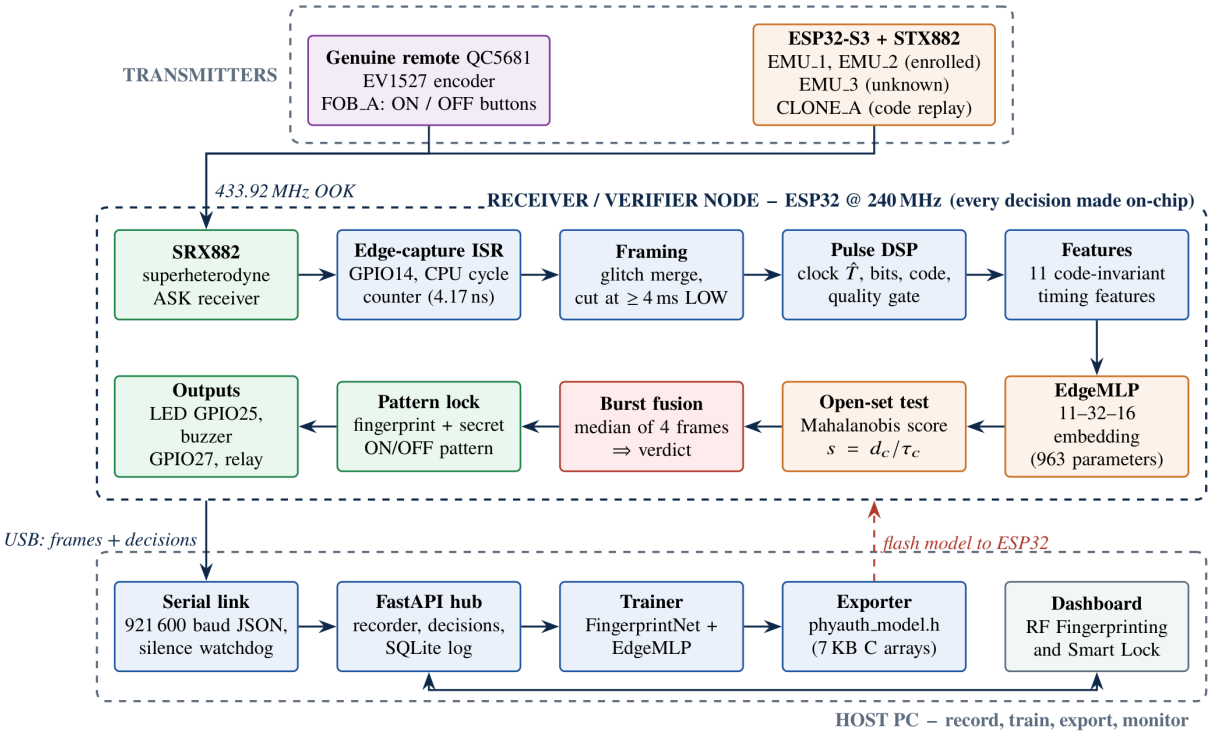


Figure 2.3: System architecture of PhyAuth-RF: transmitters, the receiver/verifier node (all decisions on-chip) and the laptop tool-chain (record, train, export, visualise).

2.4 Hardware Design

Only two ESP32 boards and one module pair were available, so the hardware was split into a receiver/verifier node and a transmitter/emulator node (Table 2.1). The receiver ESP32 adds a green status LED with a 470 Ω series resistor and a piezo buzzer for the lock.

Table 2.1: Hardware used

Item	Part	Role / connection
Receiver MCU	ESP32 DevKit (ESP32-D0WD-V3, 240 MHz, CP2102 USB-UART)	Edge timestamping, DSP, TinyML, lock FSM; USB to laptop (921 600 baud)
Receiver RF	SRX882 superheterodyne ASK receiver, 433.92 MHz	VCC, CS → 3V3; GND; DATA → GPIO14
Antennas	Helical spring antennas (433 MHz)	Soldered to SRX882 and STX882 ANT pads
Status LED	Green 5 mm LED + 470 Ω	GPIO25 → resistor → LED → GND
Buzzer	Piezo buzzer (5 V type)	GPIO27 → buzzer → GND (quiet at 3.3 V; NPN driver recommended)
Transmitter MCU	ESP32-S3-DevKitC-1 (N16R8)	RMT pulse generation (100 ns ticks), BOOT button = manual press
Transmitter RF	STX882 ASK transmitter, 433.92 MHz	DATA ← GPIO4; VCC → 3V3; GND
Genuine device	QC5681 two-button remote, EV1527, 23A 12 V cell	FOB_A: ON 0x936768, OFF 0x936764

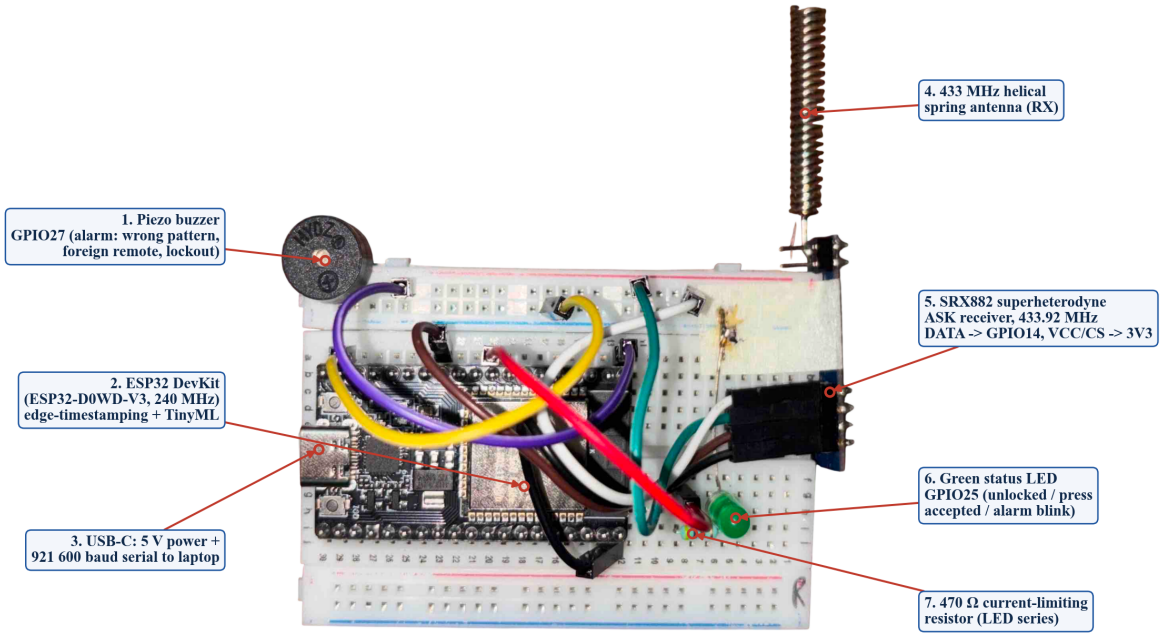


Figure 2.4: Receiver / verifier node (labelled): ESP32 DevKit, SRX882 receiver with helical antenna, green status LED with 470 Ω resistor, and piezo buzzer.

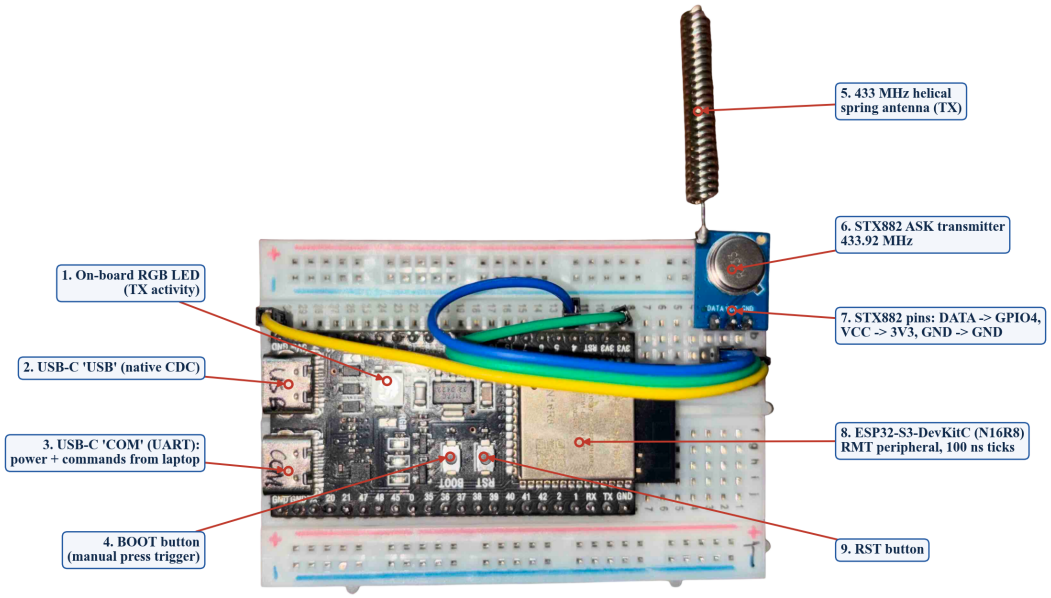


Figure 2.5: Transmitter / emulator node (labelled): ESP32-S3-DevKitC (N16R8) driving an STX882 ASK transmitter with a helical antenna.

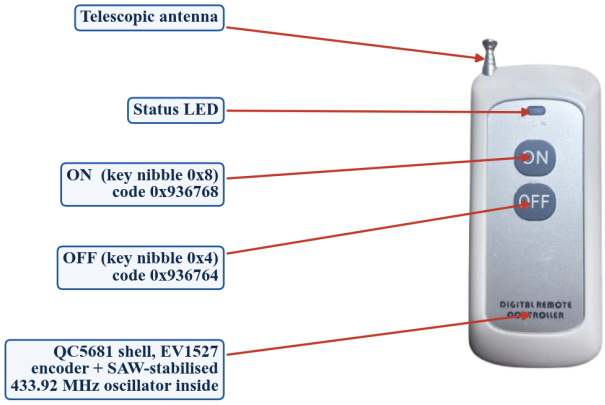


Figure 2.6: Genuine device FOB_A: QC5681 two-button EV1527 remote (front view).

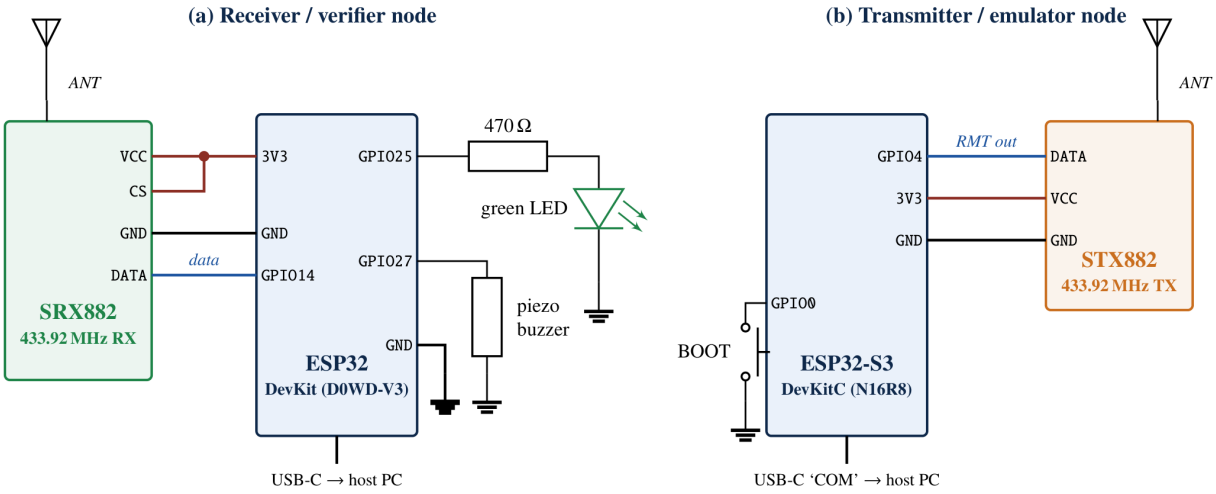


Figure 2.7: Circuit schematic of the two nodes. The SRX882 is powered from 3.3 V so that its DATA output is a safe logic level for the ESP32.

2.5 Receiver Firmware: Nanosecond Edge Capture

A GPIO interrupt on any edge of GPIO14 reads the CPU cycle counter [22] (240 MHz, 4.17 ns per count) and pushes it into a 1024-entry lock-free ring buffer; nothing else happens in the ISR. The main loop converts edges into pulse widths, merges glitches shorter than 80 µs and cuts a frame when the line stays LOW for 4 ms, keeping the preceding gap g (the sync LOW) with the frame. Frames with fewer than 16 pulses are discarded as noise. Each frame is printed as one JSON line (protocol v2), together with periodic status lines used by the laptop as a link-health heartbeat:

```
{"type":"frame","seq":1,"t_us":3921291,"g":10377700,"n":49,"d":[929400,419500,258450,1064275,...]}
{"type":"auth","verdict":"AUTHENTIC","dev":"FOB_A","code":"936768/24","score":0.81,"frames":4,...}
{"type":"lock","event":"progress","state":"LOCKED","n":3,"len":9,"fails":0,"led":0}
```

The transmitter node generates frames with the ESP32-S3 RMT peripheral (100 ns ticks) [23]. An important detail found during testing: sending each frame as a separate RMT call left a 2 ms gap between frames that corrupted the last bit; the firmware therefore builds the whole press (all frames and the closing sync HIGH) as one continuous waveform.

2.6 Pulse DSP: Distortion-Invariant Clock Recovery and Decoding

Two quantities are independent of the slicer bias δ because δ cancels inside a HIGH–LOW pair: the pair period P (which is 4T for EV1527) and the difference between long and short HIGHs. The sync gap, 31T, then fixes the number of units per pair uniquely:

$$P_k = w_{2k}^{H} + w_{2k+1}^{L} = nT, \qquad n^{*} = \{\, n \in 3..8 : |g\, n/\bar{P} - 31| \le 0.08 \cdot 31 \,\}, \qquad \hat{T} = \bar{P}/n^{*} \tag{2.4}$$

If the gap is unavailable, n is estimated from the long/short HIGH difference, n = 2P/(P − ΔH). Bits are then decided per pair ("the HIGH is the long half"), which is again immune to δ. A quality gate rejects frames whose pair-period RMS error exceeds 8 % of T or whose pulses lie more than 0.45T from the grid. For every accepted frame the residual of each pulse is

$$r_i = \frac{w_i - u_i \hat{T}}{\hat{T}}, \qquad u_i = \mathrm{round}(w_i/\hat{T}) \tag{2.5}$$

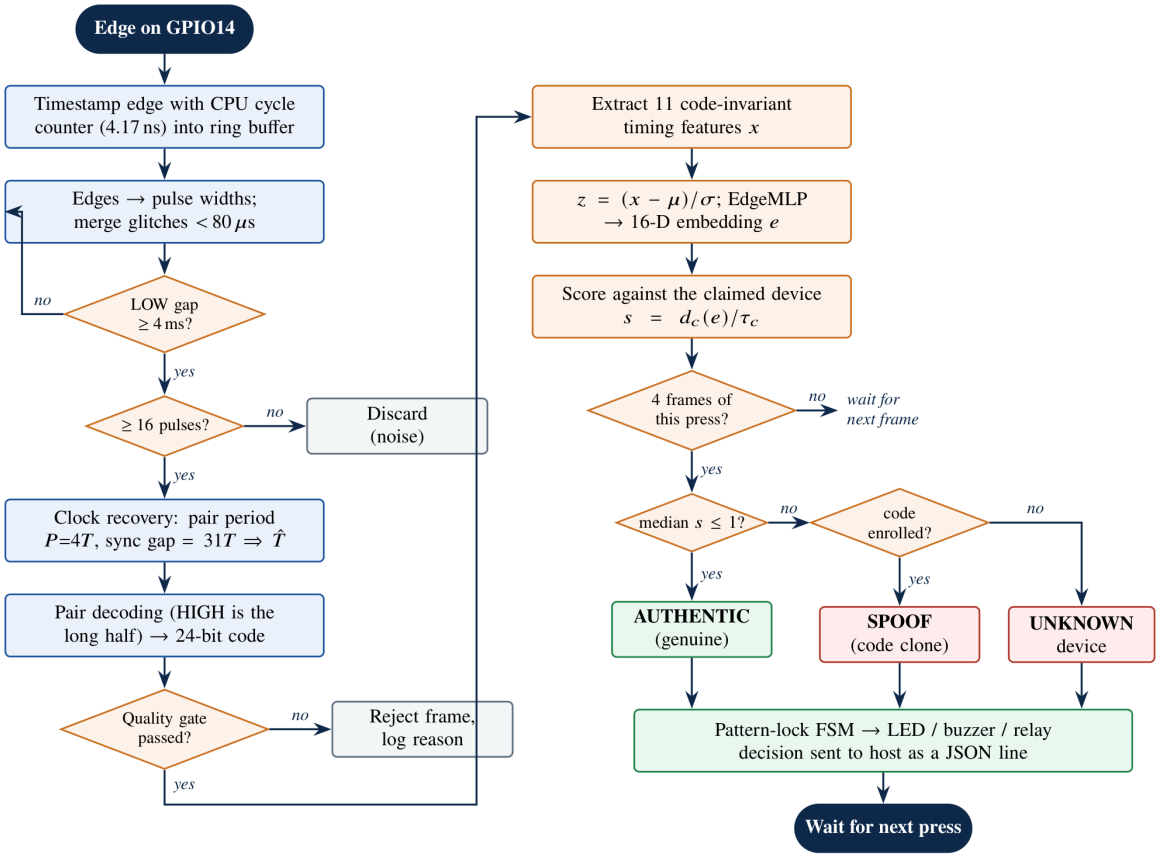


Figure 2.8: Flowchart of the processing chain from a radio edge to an authentication decision (identical in the ESP32 C code and in Python).

2.7 Physical Fingerprint Features

Eleven features are computed from the residuals and timing of each frame (Table 2.2). None of them depends on the transmitted bits, so the same model works for every button and code of a device.

Table 2.2: The eleven code-invariant timing features

#	Feature	Definition	Physical origin
0	clock_T_us	Recovered symbol period $\hat{T}$	Encoder RC/crystal timebase frequency
1	pair_jitter_ppm	Std. of bit-period error (ppm)	Oscillator cycle-to-cycle jitter
2	duty_short_high	Mean (H − T)/T, short HIGHs	PA turn-on/off asymmetry + slicer
3	duty_long_high	Mean (H − 3T)/T, long HIGHs	Edge asymmetry at long symbols
4	high_jitter	Std. of HIGH residuals	Amplitude/edge noise of the PA
5	first_edge_resid	Residual of the first HIGH	Turn-on transient of the transmitter
6	drift_ppm_per_bit	Slope of bit-period error vs bit index	Oscillator warm-up chirp
7	sync_ratio	Preceding sync gap g / $\hat{T}$	Encoder sync timing
8	tail_high_resid	Residual of the trailing sync HIGH	Turn-off behaviour
9	duty_short_low	Mean (L − T)/T, short LOWs	Complementary edge asymmetry
10	duty_long_low	Mean (L − 3T)/T, long LOWs	Complementary edge asymmetry

2.8 Learning: Embedding Models

Two models are trained on the same frames. The **host model** (FingerprintNet, 21 427 parameters) fuses a 1D ResNet [19] over the per-bit timing-error sequence (3 × 32) with an MLP over the 11 features. The **edge model** (EdgeMLP, 963 parameters) uses only the features, so it can run on the ESP32:

$$z = \frac{x-\mu}{\sigma}, \qquad e = \frac{W_2\,\mathrm{ReLU}(W_1 z + b_1) + b_2}{\|W_2\,\mathrm{ReLU}(W_1 z + b_1) + b_2\|_2} \in \mathbb{R}^{16} \tag{2.6}$$

Both are trained with label-smoothed cross-entropy plus a batch-hard triplet loss [15], [16] that pulls frames of one device together and pushes other devices at least a margin m = 0.3 away on the unit sphere:

$$\mathcal{L} = \mathrm{CE}_{0.05}(y, 8\,W_c e) + \frac{1}{B}\sum_a \max\left(0,\ \max_p \|e_a - e_p\|^2 - \min_n \|e_a - e_n\|^2 + 0.3\right) \tag{2.7}$$

Training uses AdamW [20] with cosine learning-rate decay for 60 epochs and a class-balanced sampler. Augmentation imitates the variation seen between days and distances: T scaled by N(1, 0.008), HIGH jitter scaled by exp(N(0, 0.35)) and drift perturbed by N(0, 150) ppm/bit. Splits are *temporal* (later presses are held out), never random, so test frames come from presses the model has not seen. The OFF button of the remote is recorded as a *variant* (FOB_A_OFF) and merged into FOB_A for training, so the fingerprint is learned per transmitter rather than per button.

2.9 Open-Set Authenticator and Burst Fusion

For each enrolled device c the centroid and covariance of its validation embeddings are estimated. Because only tens to hundreds of frames are available in 16 dimensions, the covariance is shrunk towards a scaled identity (Ledoit–Wolf [12]):

$$\Sigma_c = (1-\lambda)S_c + \lambda\frac{\mathrm{tr}(S_c)}{d} I, \qquad d_c(e) = \sqrt{(e-\mu_c)^T \Sigma_c^{-1} (e-\mu_c)} \tag{2.8}$$

The threshold is set from genuine data only (one-class), as the 97th percentile of genuine validation distances times a drift margin of 1.3; attacker data are never used for tuning, so reported attack results are not optimistic. Each frame gets a normalised score and a press is decided on the median of its first four frames:

$$\tau_c = 1.3 \cdot Q_{0.97}\left(d_c^{genuine}\right), \qquad s_j = \frac{d_c(e_j)}{\tau_c}, \qquad \text{AUTHENTIC} \Leftrightarrow \text{median}(s_1, \ldots, s_4) \le 1 \tag{2.9}$$

If the decoded code belongs to an enrolled device and the score fails, the verdict is SPOOF SUSPECTED (code clone). If the code is unknown, the nearest profile is reported and the verdict is UNKNOWN DEVICE. A press with fewer than three good frames is not decided.

Liveness (start-up drift). A remote is unpowered until pressed, so its oscillator settles during the first frames; an always-on crystal transmitter shows no such drift. The per-press drift

$$\rho = \frac{\hat{T}_{first} - \hat{T}_{last}}{\hat{T}_{last}} \cdot 10^6 \text{ ppm} \tag{2.10}$$

is profiled per device, and the check is switched on only if the accept window (median ± 5 sd) excludes zero by at least 300 ppm. Otherwise it stays off rather than rejecting genuine presses (see Section 4.7).

2.10 Edge Deployment

The exporter writes the scaler, EdgeMLP weights, per-device centroids, inverse covariances, thresholds, codes and liveness profiles into phyauth_model.h (7 012 bytes of constant arrays). The C implementation of the DSP and the model (phyauth_dsp.c) mirrors the Python code line by line; a parity test compiles it on the host and checks that both produce the same features, scores and verdicts on recorded frames. Per frame the on-chip model needs about 1.1 k multiply–accumulates (11×32 + 32×16 + 16×16), negligible beside the 42.9 ms frame time. The firmware builds for both the ESP32 and the ESP32-S3.

2.11 Two-Factor Smart Lock

The lock is a pure C state machine (phyauth_lock.c) driven by decided presses (Figure 2.9). Factor 1 is the RF fingerprint: a press counts as genuine only if the authenticator accepts it as the lock's owner. Factor 2 is a secret sequence of ON/OFF presses, 4–12 long (the demonstration used 9). Design rules:

- Every press advances the attempt; the result is judged only at full length, so an observer cannot learn which press was wrong.
- A press rejected by the fingerprint (clone, unknown or unclear) is counted but spoils the attempt: a clone can never unlock even with the right pattern.
- Presses from another enrolled device are ignored; 30 s of inactivity abandons an unfinished attempt without penalty.
- Three consecutive failures cause a lockout of 30 s that doubles with every further failure (up to 8×).
- After unlocking, entering the pattern again locks; auto-relock after 15 s (configurable) or on command.
- Learn mode: the owner enters the new pattern twice on the remote; it is stored in NVS flash and survives reboots and model updates.

- Outputs: green LED on while unlocked, short flash per accepted press, slow blink in learn mode, fast blink with buzzer on failure or lockout; a relay output (GPIO33) follows the unlocked state.

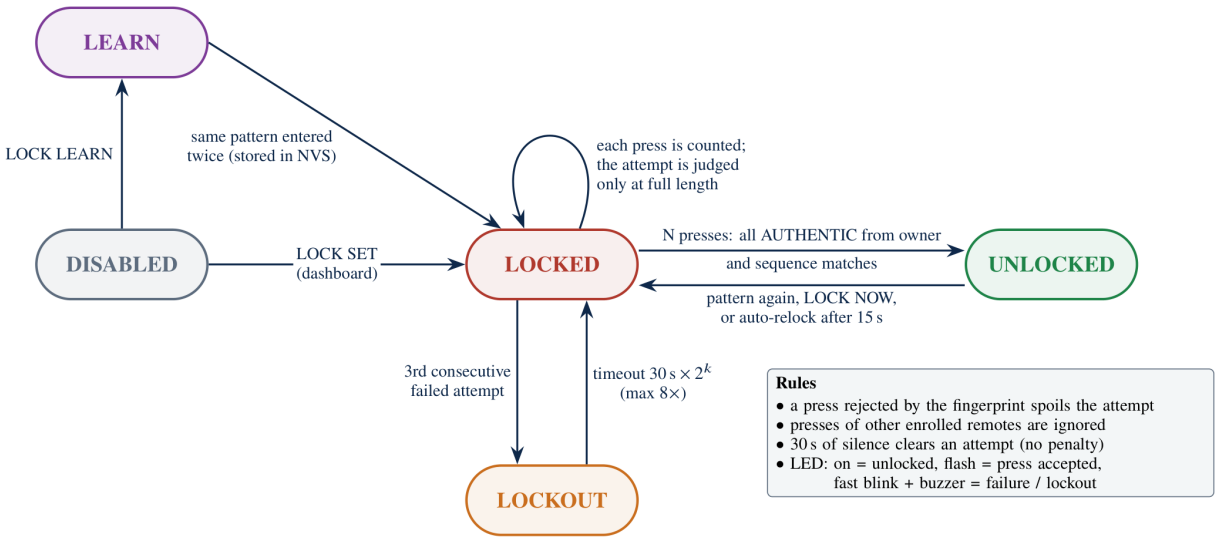


Figure 2.9: State machine of the two-factor pattern lock running on the receiver ESP32.

2.12 Software Stack and Dashboard

The laptop runs a FastAPI back-end (serial link, recorder, trainer, exporter, SQLite event log, REST + WebSocket) and a Next.js dashboard with two workspaces: **RF Fingerprinting** (Live, Capture, Model lab & deploy, Attack lab, Devices, Hardware) and **Smart Lock** (padlock view, two factor cards, timeline, settings). The serial reader has a watchdog: the receiver sends a status line every 2 s, so 8 s of silence reopens the port automatically (this fixed a stall after re-flashing). In total about 10 600 lines of Python, C/C++ and TypeScript were written, with 35 automated tests.

2.13 Experimental Methodology

Table 2.3 lists the transmitters. Recordings were made at 0.75 m, 1.0 m and 1.25 m on three sessions (s01 on 17 Oct, s02/s03 on 21 Oct 2026), holding each button for a normal press.

Table 2.3: Transmitters used in the experiments

ID	Hardware	Role	Code	Notes
FOB_A	QC5681 remote, ON button	Genuine, enrolled	936768/24	T ≈ 335 µs
FOB_A_OFF	QC5681 remote, OFF button	Variant of FOB_A	936764/24	Merged into FOB_A
EMU_1	ESP32-S3 + STX882, profile 1	Enrolled (emulated)	5a3c91/24	T 318 µs, 2500 ppm jitter
EMU_2	ESP32-S3 + STX882, profile 2	Enrolled (emulated)	1e77c4/24	T 331 µs, 3500 ppm jitter
EMU_3	ESP32-S3 + STX882, profile 3	Unknown transmitter	9b0d2a/24	Never enrolled
CLONE_A	ESP32-S3 + STX882, clean	Attacker: replays FOB_A	936768/24	Same code and T as FOB_A

Metrics: FAR = accepted impostor decisions / impostor decisions; FRR = rejected genuine decisions / genuine decisions; EER is the error rate where FAR = FRR when the threshold is swept;

AUC is the area under the ROC curve. They are reported per frame and per burst (press). Attack rejection is the fraction of clone or unknown bursts that are not accepted.

Honest labelling of emulated devices. EMU_1 and EMU_2 differ from each other by *programmed* timing (period, jitter, skew), which is a weaker difference than between two real remotes. Results that separate EMU_1 from EMU_2 are therefore a functional test of the pipeline; the security-relevant results are those for the genuine remote against CLONE_A and EMU_3, which are physically different transmitters.

2.14 Implementation Plan

Table 2.4: Implementation plan

Phase	Work	Outcome
1	Firmware for edge capture and RMT transmission; serial protocol	Raw frames from both boards
2	Bench tests of distance, antenna, receiver distortion	0.75–1.25 m window; distortion model
3	Distortion-invariant clock and pair decoder; regression tests	Remote decoded at all distances
4	Features, models, open-set authenticator, evaluation	Model + metrics report
5	C port, exporter, parity test, on-chip decisions	Authentication on the ESP32
6	Two-factor lock, LED/buzzer, learn mode, flash storage	Working lock prototype
7	Dashboard (two workspaces), watchdog, data collection, retraining	Final system and this report

CHAPTER 3

Innovation & Feasible Solution

3.1 Novel Aspects of the Solution

- **RF fingerprinting without an ADC or SDR.** All features come from edge times measured with the CPU cycle counter (4.17 ns) behind a $1 envelope receiver.
- **Distortion-invariant clock recovery.** Using the pair period and the 31T sync gap removes the receiver's slicer bias; the conventional histogram method locked onto 2T/3 and decoded nothing from the real remote.
- **Code-invariant fingerprint.** Features never use the bit values, so one model covers both buttons; button variants are merged per transmitter.
- **On-device open-set authentication.** A 963-parameter network plus Mahalanobis profiles (7 KB) decides each press on the ESP32 and rejects transmitters it has never seen; Python and C are kept identical by a parity test.
- **Two factors on one unmodified remote.** The smart lock combines the physical fingerprint (have) with a press pattern (know); a cloned code with the correct pattern still fails, and the pattern is never revealed press by press.
- **Self-validating liveness.** The start-up drift check enables itself only when it is statistically safe, avoiding false rejections.
- **Complete reproducible tool-chain.** Record → train → evaluate → export → flash → live dashboard, with 35 automated tests and an emulator for controlled attacks.

3.2 Comparison with Conventional Approaches

Table 3.1: Qualitative comparison

Aspect	Typical SDR-based RFFI	Fixed-code receiver	PhyAuth-RF
Receiver front end	SDR with 10–100 MS/s I/Q ADC	OOK module	OOK module (SRX882)
Measured quantity	Raw I/Q waveform	Bits only	Edge times at 4.17 ns
Processing	PC/GPU	MCU decoder	ESP32 (on-chip) + optional PC
Detects code clone	Yes	No	Yes (30/30 bursts)
Rejects unknown device	Depends on method	No	Yes, open-set (30/30 bursts)
User secret factor	No	No	Yes, ON/OFF pattern
Approx. receiver cost	Hundreds to thousands of USD	< Rs. 200	≈ Rs. 650

3.3 Technical Feasibility

- **Timing resolution:** 4.17 ns per count vs. a symbol of 335 µs, i.e. 1 part in 80 000; the fingerprint features vary at the 10^4 ppm level and are easily resolved.
- **Computation:** about 1.1 k MACs and an O(pulses) DSP pass per frame on a 240 MHz core; the decision latency (≈ 172 ms for four frames) is set by the air time of the frames, not by computation.
- **Memory:** model header 7 012 bytes; ring buffer 4 KB; lock state < 100 bytes; pattern in NVS flash.

- **Interfaces:** a single GPIO input for the receiver; LED, buzzer and relay outputs; USB serial only for monitoring and training. The node works stand-alone once flashed.
- **Range:** a reliable decoding window of 0.75–1.25 m with the spring antennas (Section 4.5); enough for a door-side reader, and extendable with a better antenna or a receiver with AGC.

3.4 Economic Feasibility

Table 3.2: Bill of materials (approximate Indian retail prices, October 2026)

Component	Qty	Approx. cost (Rs.)
ESP32 DevKit (receiver)	1	400
ESP32-S3-DevKitC N16R8 (transmitter / emulator)	1	800
SRX882 + STX882 433 MHz module pair with spring antennas	1 set	300
QC5681 two-button EV1527 remote	1	250
Breadboards, jumper wires	2 + set	240
Green LED, 470 Ω resistor, piezo buzzer	1 each	30
Total development kit		**≈ 2 020**
Deployable verifier only (ESP32 + SRX882 + LED/buzzer)		**≈ 600–650**

A deployable verifier costs about as much as the receiver it would replace plus a microcontroller, and protects existing remotes without changing them, so no new key fobs need to be bought or discarded.

3.5 Safety Considerations

- **Radio:** 433.92 MHz lies in the license-exempt short-range-device band (433–434.79 MHz in India); the STX882 is a low-power module and transmissions are short bursts.
- **Electrical:** all logic is 3.3 V from USB; the SRX882 is powered from 3.3 V so its DATA pin cannot exceed the ESP32's input limit; the LED current is limited by 470 Ω (≈ 3 mA).
- **Actuators:** a relay or lock driver must be electrically isolated and should fail secure; mains loads must not be wired on a breadboard.
- **Buzzer:** the 5 V buzzer draws more than a GPIO should source at full volume; an NPN transistor driver is required for production.
- **Responsible testing:** the clone emulator replays only our own remote's code in the lab; no third-party systems were targeted.

3.6 Operational Constraints

- Each remote must be enrolled (about 30–40 presses per button and distance) before use, and a new receiver needs new enrolment, because the receiver's distortion is part of every measurement.
- Fingerprints drift between days, temperatures and battery levels; recording sessions on several days and augmentation mitigate this, and re-enrolment may be needed after a battery change.
- Decoding requires 0.75–1.25 m with the current antennas; strong signals saturate the slicer and weak ones lose short pulses.

CHAPTER 4

Results & Discussion

4.1 Physical Prototype

Both nodes were built on breadboards (Figures 2.4–2.6) and run the final firmware v2.1. The final deployed model is version 20261021-224341 (trained in 17.4 s on the laptop), flashed to the receiver ESP32; the laptop runs the same version, so both deciders agree.

4.2 Dataset

Of 1 602 recorded frames, 1 442 (90.0 %) passed the quality gate. Most rejections were frames with too few pulses (typically fragments at the start or end of a press). After removing frames with a foreign code, training used 712 frames, validation 137 and testing 152 genuine frames, plus 420 impostor frames (CLONE_A and EMU_3) that were never used for training or threshold selection.

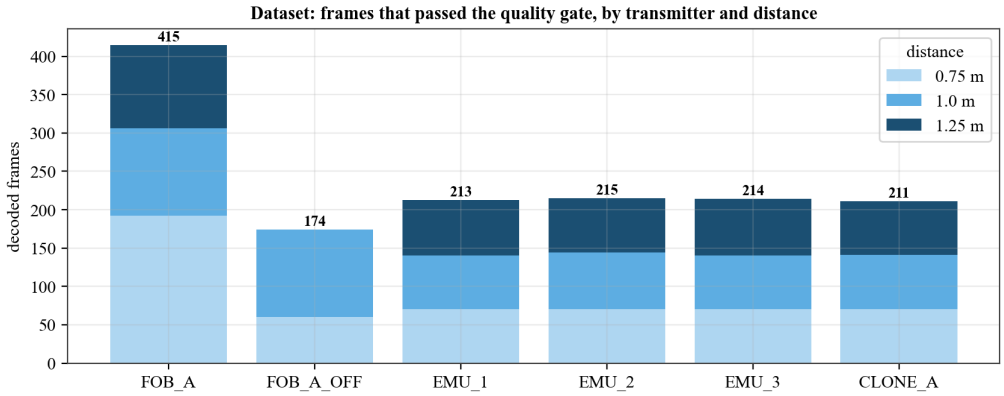


Figure 4.1: Frames that passed the quality gate, by transmitter and distance (FOB_A_OFF includes 131 frames recorded on 21 Oct to strengthen the OFF button).

4.3 Feature Space

Figure 4.2 plots two pairs of raw features. Clock period alone separates the emulated profiles and the unknown transmitter, but **CLONE_A deliberately uses the same code and the same T as FOB_A** and overlaps it in clock period; it is separated by edge-shape features (duty error, first-edge residual) that come from its different power amplifier and timebase. The FOB_A clusters spread with distance and session, which is why multi-distance enrolment matters.

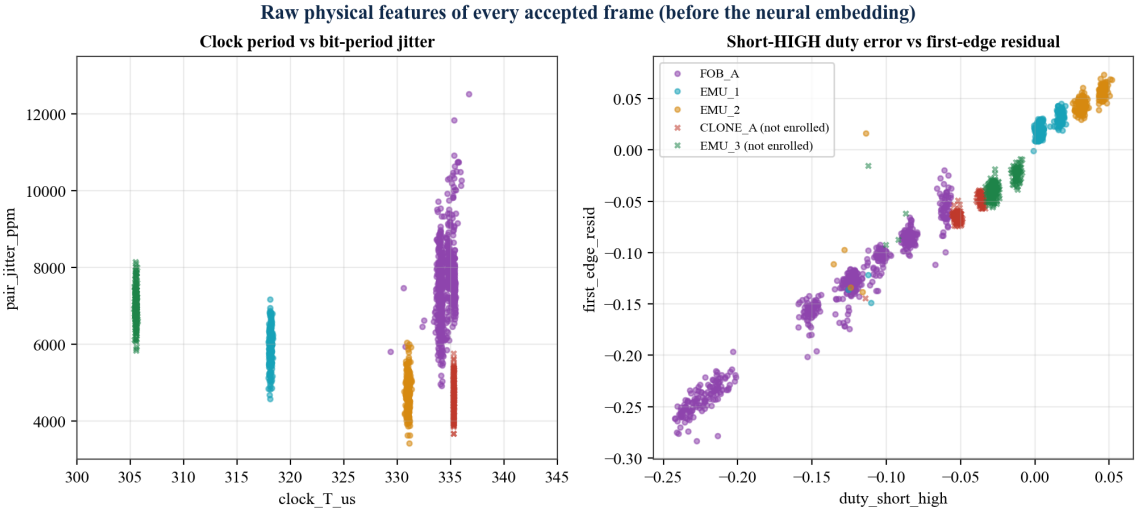


Figure 4.2: Raw physical features of every accepted frame. Left: clock period vs. bit-period jitter (13 of 1 442 frames that locked to 3T lie off-scale). Right: short-HIGH duty error vs. first-edge residual; the clone (red) is clearly apart from the genuine remote (purple).

Table 4.1: Permutation importance of the features for the edge model (test set)

Feature shuffled	Frame EER after shuffling	Increase in EER
clock_T_us	27.04 %	+26.55 %
pair_jitter_ppm	16.86 %	+16.37 %
duty_long_low	5.58 %	+5.09 %
duty_long_high	4.57 %	+4.08 %
first_edge_resid	4.57 %	+4.08 %
tail_high_resid	4.57 %	+4.08 %
duty_short_low	3.56 %	+3.07 %
duty_short_high	3.31 %	+2.82 %
high_jitter	1.39 %	+0.90 %
drift_ppm_per_bit	0.74 %	+0.26 %
sync_ratio	0.49 %	+0.00 %

Clock period and pair jitter carry most identity, followed by edge-asymmetry features. The sync ratio contributes nothing for these devices and could be removed to save computation.

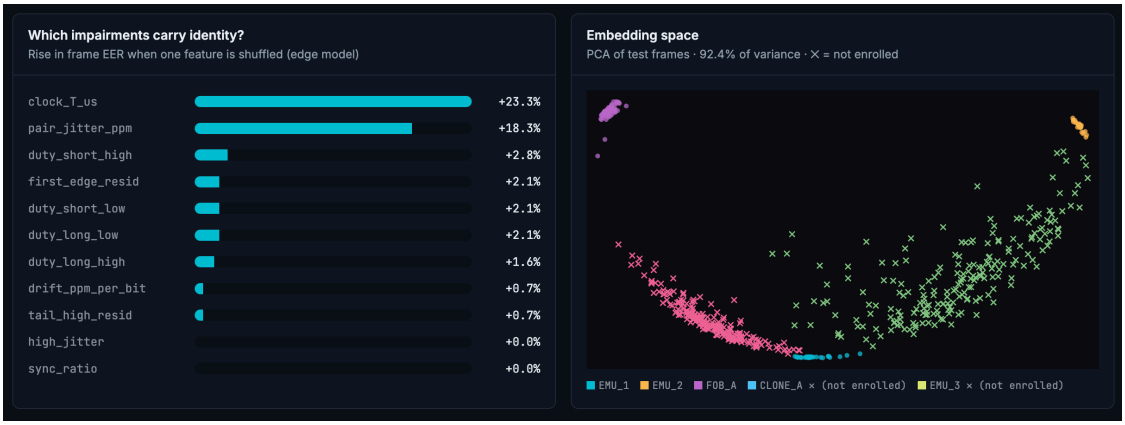


Figure 4.3: Dashboard Model lab: feature importance and PCA of the embedding space (previous model version 20261021-213950). Enrolled devices form tight clusters; the clone (×, not enrolled) lands between FOB_A and the emulators, outside every acceptance region.

4.4 Authentication Performance

Table 4.2: Results of the final model (version 20261021-224341) on held-out data

Metric	Edge model (on ESP32)	Host fusion model
Parameters / size	963 / 7 012 B header	21 427
Closed-set accuracy (152 test frames)	100 %	100 %
Frame EER / AUC	0.49 % / 0.99996	0.13 % / 1.0000
Frame FAR / FRR at operating threshold	0 % / 3.9 %	0 % / 16.4 %
Burst EER / AUC	0 % / 1.0000	0 % / 1.0000
Burst FAR / FRR (27 genuine, 234 impostor bursts)	0 % / 0 %	0 % / 0 %
Code clone CLONE_A rejected	30 / 30 bursts (100 %)	30 / 30 bursts (100 %)
Unknown EMU_3 rejected	30 / 30 bursts (100 %)	30 / 30 bursts (100 %)
Validation EER	0.36 %	0.36 %

The edge model is the better *operating* model: the host model separates classes slightly better (lower frame EER) but its one-class threshold is tighter, so it rejects more genuine single frames (16.4 % vs 3.9 %). Fusing frames removes the difference: at the burst level both make no error on this test set. With only 27 genuine test bursts the true burst FRR can still be a few percent (a 95 % upper bound of about 11 % for 0 errors in 27), so the live results below are an important complement.



Figure 4.4: Dashboard Model lab: ROC on a logarithmic false-accept axis and the closed-set confusion matrix with per-distance frame FRR (previous model version, 213950).

4.5 Distance Robustness

Figure 4.5(a) gives the single-frame FRR of the final models per distance; accuracy was 100 % everywhere. The bench window in (b) was measured by moving the remote: closer than 0.5 m the receiver saturates and the slicer bias grows, and from 1.5 m the short 1T pulses are lost. In a leave-one-distance-out test (Table 4.3), the model was trained without one distance and tested on it.

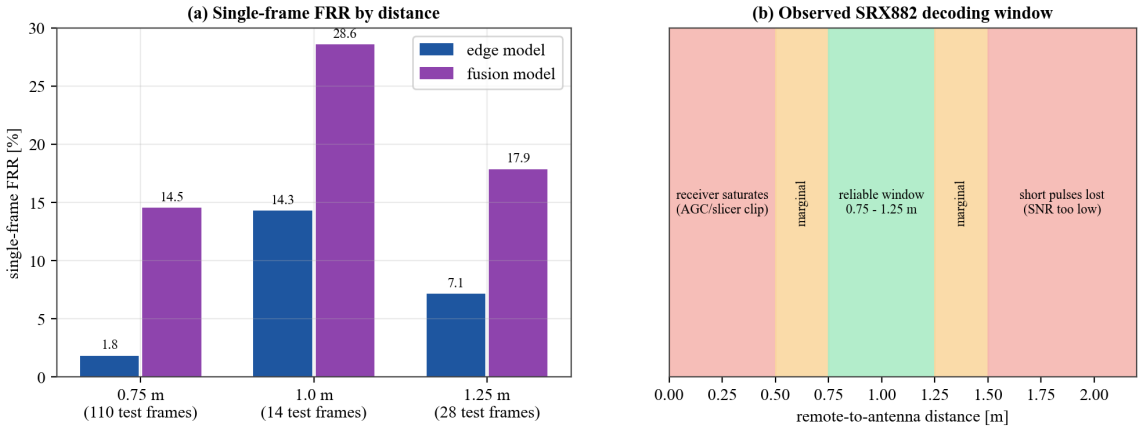


Figure 4.5: (a) Single-frame genuine rejection by distance for both models. (b) Decoding window of the SRX882 rig observed on the bench.

Table 4.3: Leave-one-distance-out test (presses; development model trained on 17 Oct data)

Held-out distance	FOB_A rejected	EMU_1 rejected	EMU_2 rejected	Clones rejected	Unknown rejected
0.75 m	0/10	0/10	0/10	10/10	10/10
1.0 m	0/10	0/10	0/10	10/10	10/10
1.25 m	1/10	0/10	0/10	10/10	10/10

Only one genuine press out of 90 was rejected when its distance had never been seen in training, while every clone and unknown press was rejected at every distance.

4.6 Burst Fusion and Latency

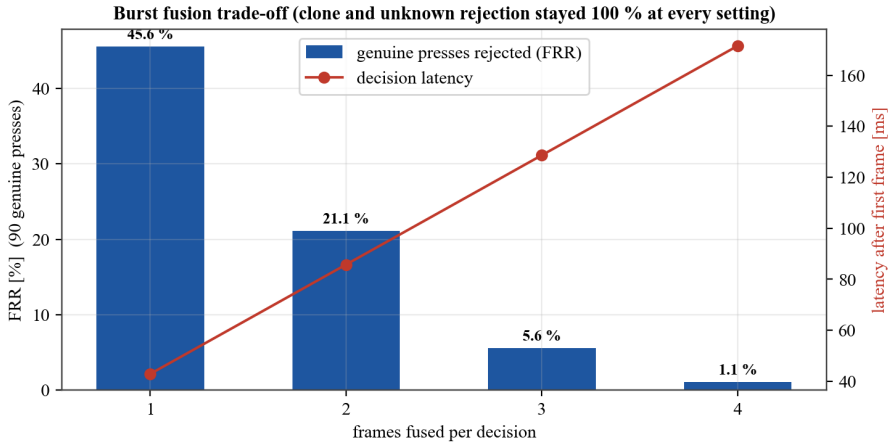


Figure 4.6: Genuine rejection rate and decision latency against the number of frames fused per press (90 genuine presses).

A single frame is a noisy measurement (45.6 % of genuine presses would be rejected), but the median of four frames brings the FRR to 1.1 % at a latency of 171.6 ms after the first frame, still well within a normal button press. Clone and unknown rejection stayed at 100 % for every setting, so four frames were chosen; the firmware decides on three frames if a press is shorter.

4.7 Cross-Session Check and Liveness

Table 4.4: Presses from a later session (21 Oct) against a model trained on 17 Oct data

Device	Presses	Accepted	Matched profile	Median score	By distance
FOB_A (ON)	31	31	FOB_A	0.63	0.75 m: 10/10, 1.0 m: 11/11, 1.25 m: 10/10
FOB_A_OFF	10	10	FOB_A	0.66	1.0 m: 10/10

All presses of the genuine remote were accepted on a different day, and OFF presses (a different code) matched the FOB_A profile without being enrolled separately, confirming that the fingerprint belongs to the transmitter, not the code. During live lock use, however, OFF presses held differently in the hand scored close to the threshold (1.0–1.6); 131 further OFF frames were therefore recorded and the model retrained (final model). Its live re-test is part of the next demonstration.

Table 4.5: Start-up drift per press (liveness profiles)

Device	Presses	Median drift (ppm)	Robust sd (ppm)	Check active?
FOB_A	86	-1032	526	no – no consistent start-up drift
EMU_1	26	74	359	no – no consistent start-up drift
EMU_2	26	-4	410	no – no consistent start-up drift
CLONE_A (impostor)	30	5.1	–	–

The genuine remote does show start-up drift (median −1031 ppm) while the always-on clone shows almost none (5 ppm), which confirms the physical idea. Its spread between presses (sd 526 ppm) is too large for a safe five-sigma window, so the check correctly stayed off: enabling it would have rejected genuine presses. A remote with a steadier oscillator, or more frames per press, could enable it.

4.8 Live System and Dashboard

Figure 4.7 and Figure 4.8 show live decisions. A genuine FOB_A press is accepted on the ESP32 with score 0.91 and on the laptop with 0.82. An EMU_1 press scored 1.08 / 1.06 and was flagged as a spoof: this is a **false rejection** of a borderline emulated press, consistent with the few-percent burst FRR discussed above; the next press was accepted. Figure 4.9 shows the decision log, where both deciders agree on every press.

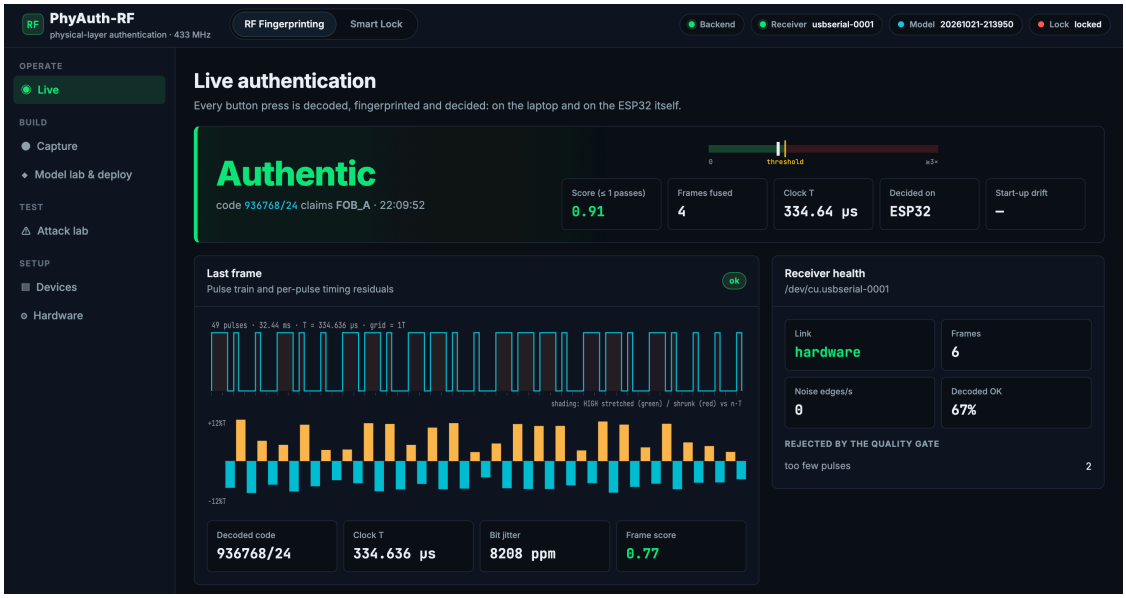


Figure 4.7: Live authentication: genuine remote accepted on the ESP32 (score 0.91, 4 frames, T = 334.64 µs), with the last frame's pulse train and timing residuals.

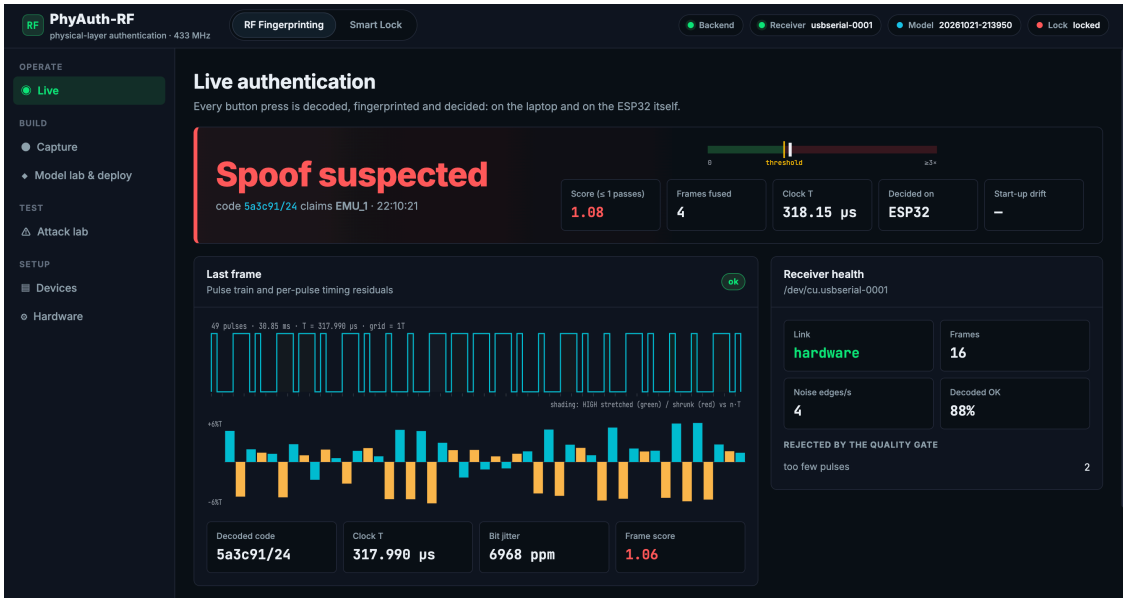


Figure 4.8: Live authentication: a borderline EMU_1 press (score 1.08 > 1) flagged as spoof suspected – an example of a false rejection.



Figure 4.9: Decision log: verdict, device, code, score, frames and which decider (ESP32 or laptop) made the decision.

4.9 Smart Lock Demonstration

The lock was set to a 9-press ON/OFF pattern owned by FOB_A. Entering the pattern on the genuine remote unlocked the ESP32 and the dashboard padlock opened (Figure 4.11); the LED followed the state and the lock re-armed automatically after 15 s. The lock logic is verified by 10 host unit tests on the same C source (Table 4.6).

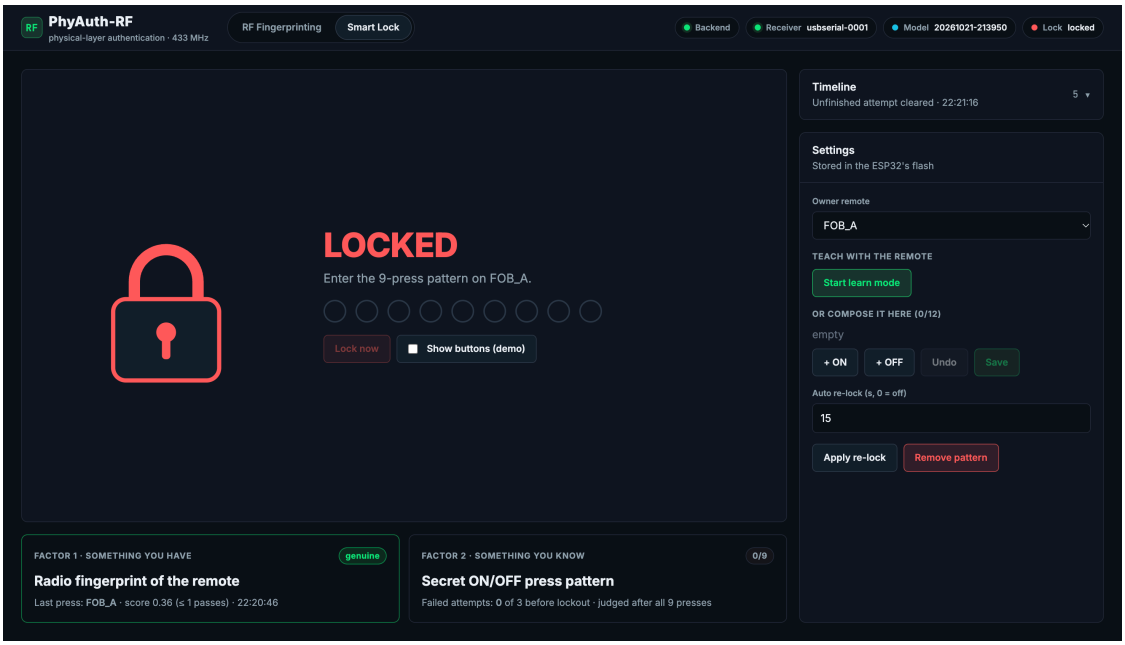


Figure 4.10: Smart Lock workspace, locked: padlock, progress dots for the 9-press pattern, the two factor cards and the settings stored in the ESP32's flash.

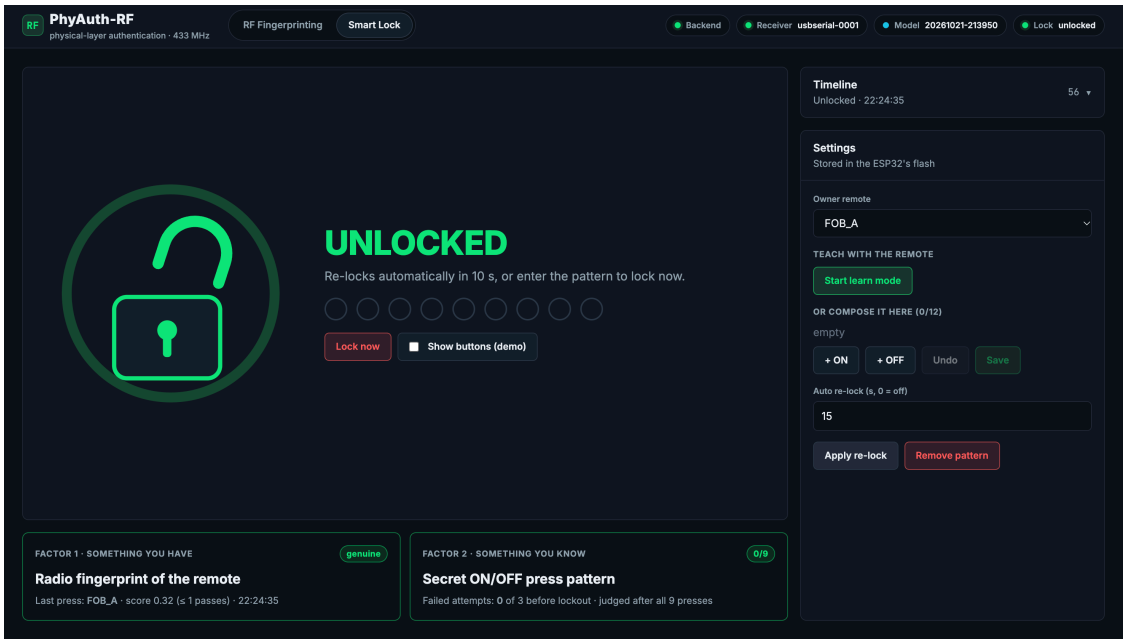


Figure 4.11: Smart Lock workspace after the correct pattern on the genuine remote: UNLOCKED, auto-relock countdown, factor 1 'genuine' (score 0.32).

Table 4.6: Unit tests of the lock state machine (all 35 project tests pass)

Test (tests/test_lock.py)	Checks	Result
correct_pattern_unlocks_and_relocks	Right pattern unlocks; same pattern locks again	pass
wrong_pattern_fails_without_revealing_position	No early failure signal	pass
rejected_press_is_counted_and_fails_once_at_the_end	Foreign press spoils attempt; one failure	pass
other_enrolled_device_is_ignored	Other enrolled remotes cannot interfere	pass
three_failures_cause_lockout_that_expires_and_grows	Lockout 30 s, doubling	pass
abandoned_attempt_is_cleared_without_failure	Idle timeout without penalty	pass
auto_relock	Relocks after the configured time	pass
learn_mode_needs_matching_confirmation	Pattern must be entered twice	pass
clone_presses_can_never_unlock	Correct pattern from a clone fails	pass
learn_rejects_too_short_and_foreign	Minimum length; foreign presses ignored	pass

4.10 Comparative Benchmarking

Direct numerical comparison with the literature is limited because the signals, devices and receivers differ. Qualitatively, laboratory systems [2], [3], [7] report very high accuracy among many devices of the same model using I/Q or oscilloscope captures; PhyAuth-RF reaches zero burst errors on its test set with a receiver costing a few hundred rupees and with the full decision on a microcontroller, but for far fewer devices. Compared with a standard fixed-code receiver, which accepts 100 % of replayed codes, PhyAuth-RF accepted none of 30 replay bursts.

4.11 Engineering Problems Solved

Table 4.7: Problems found during development and their fixes

Problem observed	Root cause	Fix
Real remote decoded 0 frames	SRX882 slicer bias up to 0.33T; clock locked to 2T/3	Pair-period + sync-gap clock, pair decoding, regression tests
Emulated frames had a wrong last bit	2 ms RMT latency between per-frame calls	Whole press as one RMT waveform with closing sync
Exported header did not compile	Float literal written as '1f'	Exporter writes '1.0f'; parity test
Same press both AUTHENTIC and SPOOF	ESP32 and laptop ran different model versions	Flash the latest export; version shown in top bar
Dashboard froze after re-flash	Serial reader stuck on a stale port handle	8 s silence watchdog reopens the port
OFF presses near threshold	Fewer OFF recordings, different grip	131 more OFF frames, retrained
Green LED only blinked	LED part is a self-flashing type	Use a plain LED (GPIO verified HIGH)
Buzzer silent	5 V buzzer on a 3.3 V GPIO	NPN transistor driver (planned)

4.12 Limitations and Threats to Validity

- One genuine remote; the emulated devices differ by programmed timing. A study with several remotes of the same model is needed to quantify inter-device separability.
- Small test sets (27 genuine and 60 attack bursts); zero errors does not mean a zero error rate.
- The fingerprint is measured through one specific receiver; moving the model to another receiver requires re-enrolment.
- Data were collected over two days indoors; temperature and battery ageing were not studied.
- A sophisticated attacker with a programmable waveform source could try to imitate the measured timing.

CHAPTER 5

Sustainable Development Goals (SDGs) & Societal Impact

5.1 Alignment with the UN Sustainable Development Goals

The 2030 Agenda [25] defines 17 goals. Table 5.1 maps the project to the most relevant goals and targets.

Table 5.1: Mapping to SDGs

SDG	Relevant target	Contribution of PhyAuth-RF
SDG 9: Industry, Innovation and Infrastructure	9.1 resilient infrastructure; 9.5 research and innovation; 9.c ICT access	Low-cost security layer for existing wireless control infrastructure; open, reproducible research tool-chain built on commodity parts
SDG 11: Sustainable Cities and Communities	11.1 safe, adequate housing; 11.7 safe public spaces	Hardens gates, garages and alarm panels of homes and community buildings against replay and cloning
SDG 12: Responsible Consumption and Production	12.5 reduce waste generation	Upgrades security of remotes already in use instead of replacing them with new rolling-code devices, avoiding e-waste
SDG 7: Affordable and Clean Energy	7.3 energy efficiency	Verifier runs on a < 1 W microcontroller instead of a PC + SDR drawing tens of watts; suitable for battery or solar power
SDG 4: Quality Education	4.4 relevant technical skills	A complete, low-cost teaching platform for digital communication, DSP, ML and embedded security

5.2 Benefits to Society

- **Safety and security of homes and small businesses:** cheap remotes are used where commercial access-control systems are unaffordable; adding a hardware-identity check closes the simplest attack, code replay.
- **Affordability:** about Rs. 650 per protected receiver makes the technique viable in developing regions and rural installations.
- **Retrofit, not replace:** users keep their remotes; only the receiver side changes, which also reduces cost and waste.
- **Two-factor convenience:** the pattern lock adds a knowledge factor without a keypad, phone or network connection, so it works offline and during network outages.
- **Education and awareness:** the Attack lab demonstrates, safely and on one's own devices, why fixed codes are insecure.

5.3 Environmental and Ethical Considerations

The system uses existing hardware, consumes little power and adds no batteries. Only timing data of the user's own devices are stored; no personal or location data are collected. The attack emulator is dual-use and is restricted to the user's own remote in a laboratory; the report deliberately describes it only at the level needed to evaluate the defence.

CHAPTER 6

Conclusion & References

6.1 Conclusion

PhyAuth-RF shows that physical-layer authentication of 433 MHz remotes is possible with a $5 microcontroller and a $1 receiver. The main outcomes are:

- Reliable decoding of a real EV1527 remote through severe receiver distortion, using a distortion-invariant clock recovery derived from the line code itself.
- Eleven code-invariant timing features measured at 4.17 ns that separate the genuine remote from a transmitter replaying its exact code.
- An on-chip open-set authenticator (963 parameters, 7 KB) with 100 % accuracy, frame EER 0.49 %, burst EER 0 % and 30/30 clone and 30/30 unknown bursts rejected on held-out data.
- A two-factor smart lock on the same ESP32 that requires both the genuine remote and a secret ON/OFF pattern, with lockout, learn mode, flash storage and LED/buzzer indication, demonstrated on hardware and covered by unit tests.
- A complete open tool-chain (firmware, DSP, training, export, dashboard) with 35 automated tests, and a documented account of the hardware's limits.

6.2 Future Work

- Enrol several remotes of the same model and collect data over weeks, temperatures and battery levels.
- Add an NPN buzzer driver, a relay-driven lock actuator and an enclosure; power the verifier from a battery.
- Better antennas and a receiver with AGC to widen the working range; a second receiver for diversity.
- Online adaptation of the profiles from accepted presses to follow slow drift.
- Extend the method to other OOK encoders (PT2262, HT12E) and to LoRa/BLE timing.

6.3 References

[1] T. Kohno, A. Broido, and K. C. Claffy, “Remote physical device fingerprinting,” *IEEE Trans. Dependable Secure Comput.*, vol. 2, no. 2, pp. 93–108, 2005.

[2] V. Brik, S. Banerjee, M. Gruteser, and S. Oh, “Wireless device identification with radiometric signatures,” in *Proc. ACM MobiCom*, 2008, pp. 116–127.

[3] B. Danev and S. Capkun, “Transient-based identification of wireless sensor nodes,” in *Proc. ACM/IEEE IPSN*, 2009, pp. 25–36.

[4] B. Danev, D. Zanetti, and S. Capkun, “On physical-layer identification of wireless devices,” *ACM Comput. Surv.*, vol. 45, no. 1, Art. 6, 2012.

[5] K. Merchant, S. Revay, G. Stantchev, and B. Nousain, “Deep learning for RF device fingerprinting in cognitive communication networks,” *IEEE J. Sel. Topics Signal Process.*, vol. 12, no. 1, pp. 160–167, 2018.

[6] S. Riyaz, K. Sankhe, S. Ioannidis, and K. Chowdhury, “Deep learning convolutional neural networks for radio identification,” *IEEE Commun. Mag.*, vol. 56, no. 9, pp. 146–152, 2018.

[7] K. Sankhe, M. Belgiovine, F. Zhou, S. Riyaz, S. Ioannidis, and K. Chowdhury, “ORACLE: Optimized radio classification through convolutional neural networks,” in *Proc. IEEE INFOCOM*, 2019, pp. 370–378.

[8] G. Shen, J. Zhang, A. Marshall, and J. R. Cavallaro, “Towards scalable and channel-robust radio frequency fingerprint identification for LoRa,” *IEEE Trans. Inf. Forensics Security*, vol. 17, pp. 774–787, 2022.

[9] A. Jagannath, J. Jagannath, and P. S. P. V. Kumar, “A comprehensive survey on radio frequency (RF) fingerprinting: Traditional approaches, deep learning, and open challenges,” *Computer Networks*, vol. 219, Art. 109455, 2022.

[10] F. D. Garcia, D. Oswald, T. Kasper, and P. Pavlidès, “Lock it and still lose it – On the (in)security of automotive remote keyless entry systems,” in *Proc. USENIX Security Symp.*, 2016, pp. 929–944.

[11] S. Kamkar, “Drive it like you hacked it: New attacks and tools to wirelessly steal cars,” presented at DEF CON 23, Las Vegas, NV, USA, 2015.

[12] O. Ledoit and M. Wolf, “A well-conditioned estimator for large-dimensional covariance matrices,” *J. Multivariate Anal.*, vol. 88, no. 2, pp. 365–411, 2004.

[13] P. C. Mahalanobis, “On the generalised distance in statistics,” *Proc. Nat. Inst. Sci. India*, vol. 2, no. 1, pp. 49–55, 1936.

[14] K. Lee, K. Lee, H. Lee, and J. Shin, “A simple unified framework for detecting out-of-distribution samples and adversarial attacks,” in *Proc. NeurIPS*, 2018.

[15] F. Schroff, D. Kalenichenko, and J. Philbin, “FaceNet: A unified embedding for face recognition and clustering,” in *Proc. IEEE CVPR*, 2015, pp. 815–823.

[16] A. Hermans, L. Beyer, and B. Leibe, “In defense of the triplet loss for person re-identification,” arXiv:1703.07737, 2017.

[17] R. David *et al.*, “TensorFlow Lite Micro: Embedded machine learning for TinyML systems,” in *Proc. MLSys*, 2021.

[18] P. Warden and D. Situnayake, *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. Sebastopol, CA, USA: O’Reilly, 2019.

[19] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proc. IEEE CVPR*, 2016, pp. 770–778.

[20] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *Proc. ICLR*, 2019.

[21] J. G. Proakis and M. Salehi, *Digital Communications*, 5th ed. New York, NY, USA: McGraw-Hill, 2008.

[22] Espressif Systems, *ESP32 Technical Reference Manual*. Shanghai, China: Espressif Systems.

[23] Espressif Systems, *ESP32-S3 Technical Reference Manual*. Shanghai, China: Espressif Systems.

[24] Silvan Chip Electronics, *EV1527 OTP Encoder Datasheet*.

[25] United Nations General Assembly, “Transforming our world: The 2030 Agenda for Sustainable Development,” A/RES/70/1, 2015.

[26] D. Hendrycks and K. Gimpel, “A baseline for detecting misclassified and out-of-distribution examples in neural networks,” in *Proc. ICLR*, 2017.

[27] Q. Xu, R. Zheng, W. Saad, and Z. Han, “Device fingerprinting in wireless networks: Challenges and opportunities,” *IEEE Commun. Surveys Tuts.*, vol. 18, no. 1, pp. 94–104, 2016.

[28] S. Jana and S. K. Kasera, “On fast and accurate detection of unauthorized wireless access points using clock skews,” *IEEE Trans. Mobile Comput.*, vol. 9, no. 3, pp. 449–462, 2010.

[29] O. Ureten and N. Serinken, “Wireless security through RF fingerprinting,” *Canadian J. Elect. Comput. Eng.*, vol. 32, no. 1, pp. 27–33, 2007.

Appendix A: Pin Map, Commands and Software Organisation

Table A.1: Pin map

Node	Pin	Function
Receiver ESP32	GPIO14	SRX882 DATA input (edge interrupt)
Receiver ESP32	GPIO25	Green LED (unlocked / accepted press / alarm blink)
Receiver ESP32	GPIO27	Buzzer (wrong pattern, foreign remote, lockout)
Receiver ESP32	GPIO26 / GPIO33	Red LED (locked) / relay input (unlocked), optional
Receiver ESP32	GPIO2	On-board blue LED (verdict indicator)
Transmitter ESP32-S3	GPIO4	STX882 DATA output (RMT)
Transmitter ESP32-S3	GPIO0 (BOOT)	Manual press button

Table A.2: Main serial commands of the receiver firmware

Command (receiver)	Effect
PING / INFO	Link check / firmware, model version and settings
MODE STREAM | EDGE | BOTH	Stream frames, decide on-chip, or both
SET GAP <us> / SET GLITCH <us> / SET MINP <n>	Framing gap, glitch merge width, minimum pulses
LIVE ON | OFF	Enable or disable the start-up-drift liveness check
LOCK STATUS / LOCK LEARN <dev>	Report lock state / start learn mode for an owner
LOCK SET <dev> <ON,OFF,...>	Store a pattern directly (4–12 presses)
LOCK CLEAR / LOCK NOW / LOCK RELOCK <s>	Remove pattern / lock immediately / auto-relock time

Table A.3: Repository layout

Path	Content
firmware/esp32_receiver/src	main.cpp (capture, protocol, outputs), phyauth_dsp.c (DSP + model), phyauth_lock.c (lock FSM), phyauth_model.h (exported model)
firmware/esp32_transmitter/src	main.cpp (RMT waveform, profiles, clone/emulation commands)
dsp/pulse_dsp.py	Clock recovery, decoding, quality gate, features (mirrored in C)
ml/	data loading and augmentation, models, triplet loss, authenticator, liveness, training, evaluation, export
acquisition/	Serial link with watchdog, protocol normalisation, simulator
backend/	FastAPI hub and routers: hardware, recordings, models, devices, events, attack lab, lock, WebSocket
frontend/	Next.js dashboard: RF Fingerprinting and Smart Lock workspaces
tests/	35 tests: DSP, ML, liveness, C parity, lock FSM, API